// QE LEARNING PATH · PHASE P05 · ~32 H

Playwright for QE

Eight tasks that take you from a first login test to migrating a Selenium suite: resilient locators, **no hard waits**, page objects, rescuing an inherited suite, tricky UI patterns, auth reuse, network mocking, traces and cross-browser runs. Playwright needs a real browser, so this page **animates the ideas** and runs the pure-Python parts; the UI tests run in your repo.

- 🧠 **Level 1 — Crack It** · the 🟢 low example
- 🚀 **Level 2 — Impress Them** · the 🔵 medium example
- 🏆 **Level 3 — Hero Mode** · the 🟣 high/complex example

🟢 Zero | 🔵 Hero | 🟣 Super Hero task levels

Z | H | SH interview-question levels

+ ADDED not in P05-ui-playwright.md — added to fill a gap

P05 — THE PHASE AT A GLANCE

🗺️ Eight tasks, ~32 hours

Goal: build fast, reliable browser tests: resilient locators, no hard waits, page objects, **rescuing an inherited suite**, tricky UI patterns, auth reuse, hybrid API+UI tests, network

mocking, debugging, cross-browser runs and **migrating from Selenium**. **Apps:** SauceDemo, The Internet, Toolshop. **Prerequisites:** P03, P04-04. **Time:** ~32 h.

ID	TASK	LEVEL	TIME	YOU DELIVER
P05-01	**First Playwright tests on SauceDemo**	Zero	3h	`tests/ui/saucedemo/test_login.py`
P05-02	**Locator strategy and data-driven login**	Zero	3h	`tests/ui/saucedemo/test_login_matrix.` `docs/locator-strategy.md`
P05-03	**Page Object Model: SauceDemo purchase end-to-end**	Hero	4h	`framework/pages/saucedemo/`, `tests/ui/saucedemo/test_checkout.py`
P05-04	**Rescue a legacy suite**	Hero	5h	`docs/legacy-rescue.md`, `tests/ui/saucedemo/test_legacy_rescue`
P05-05	**The Internet: tricky UI patterns**	Hero	4h	`tests/ui/internet/` (≥12 tests)
P05-06	**Toolshop: auth state, hybrid API+UI, network mocking**	Hero	5h	`framework/pages/toolshop/`, `tests/ui/toolshop/`

ID	TASK	LEVEL	TIME	YOU DELIVER
P05-07	**Debugging, cross-browser, mobile, visual checks and bug hunt**	Super Hero	5h	`tests/ui/test_visual.py`, `tests/ui/test_bug_hunt.py`, `docs/bugs/`, `docs/ui-debugging.md`
P05-08	**Selenium to Playwright migration**	Super Hero	3h	`docs/selenium-migration.md`, `tests/ui/migrated/`, `scripts/parity.py`

FROM THE GUIDE

Selectors: examples use locators that matched the apps when this guide was written. Apps change, so confirm with `uv run playwright codegen <url>` or the Inspector. Fixing a stale locator is a real job skill.

HOW TO USE THIS PAGE

Playwright drives a real browser, which can't start inside a web page. So the UI tests here are for **reading and copying** into your repo; run them there. What runs on this page: the visualizers, and the plain-Python parts (the money helper, the visual-diff function, the parity script). Confirm every selector with `uv run playwright codegen <url>` — the guide's own warning applies.

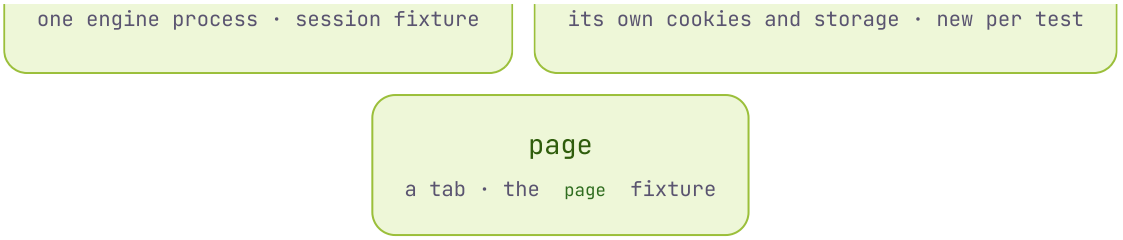
PRIMERS — FUNDAMENTALS

📚 How Playwright sees a page

Three ideas carry this phase: the **browser → context → page** hierarchy, **auto-waiting** with retrying assertions, and a **locator priority** order.

▣ Browser → context → page





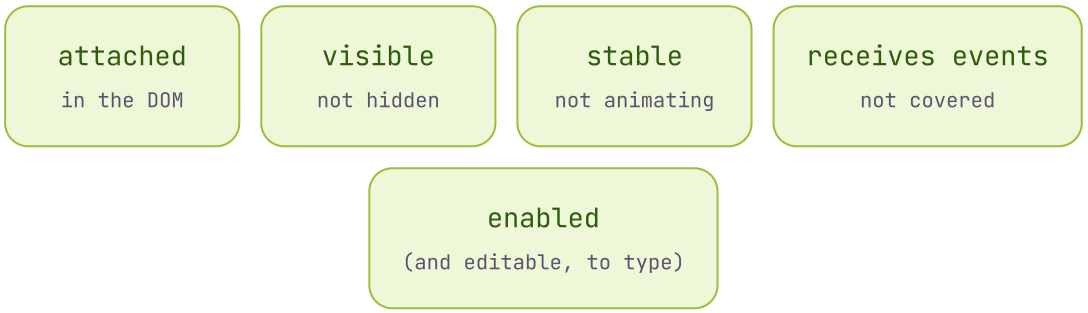
PYTEST-PLAYWRIGHT GIVES YOU	SCOPE	USE IT FOR
`page`	function	almost every test: a fresh context and tab
`context`	function	several tabs that share one login
`browser` · `browser_name`	session	building your own contexts (auth state, HTTP credentials)
`base_url`	session	`page.goto("/")` instead of full URLs
`--browser` · `--headed` · `--slowmo` · `--device`	CLI	engine, visible window, slow motion, emulation

WHY THIS MATTERS

A new context per test means no cookie or storage leaks between tests — that isolation is what lets UI tests run in parallel (P07).

⧖ Auto-waiting and web-first assertions

Before every action, Playwright waits until the element passes its **actionability checks**. Assertions written with `expect` retry the same way until they pass or time out (5 s by default).

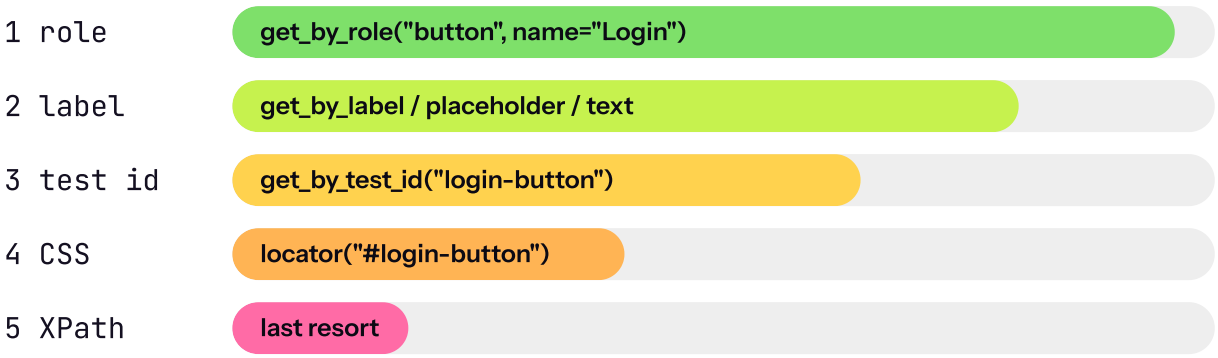


⚠ THE RULE

Use `expect` for anything on the page; use plain `assert` only on values you have already computed. And never `time.sleep` or `wait_for_timeout` in a test.

Locator priority

MOST RESILIENT FIRST



Role and label locators mirror what users and assistive tech see, so they survive markup refactors — and they fail when an element loses its accessible name, which is an accessibility bug worth catching. Try it in the P05-02 playground.

TOOLKIT — TECHNIQUES

🧰 Eight moves you'll reuse

Most UI tests are a handful of these.

🎯

Role + name

Find elements the way users do.

When: your first choice, every time.

```
page.get_by_role("button",
name="Login").click()
```

POWERS → **P05-01 · P05-02**

🔍

Scope with filter

The button inside *this* card.

When: the same control repeats.

```
card =
page.locator(".inventory_item").filter
(has_text="Sauce Labs Backpack")
card.get_by_role("button", name="Add
to cart").click()
```

POWERS → **P05-02 · P05-03**

⏳

Retrying expect

Waits and checks in one line.

When: any assertion about the page.

```
expect(page.locator(".title")).to_have
_text("Products")   # retries
```

POWERS → **every task**

🪟

frame_locator

Reach into iframes, chain for nesting.

When: the element lives in another document.

```
page.frame_locator("frame[name='frame-
top']").locator("#content")
```

POWERS → **P05-05 · P05-08**

Expect, then trigger

Register before the event can fire.

When: downloads, popups, dialogs, API responses.

```
with page.expect_download() as info:
# register first…
    link.click()
# …then trigger
```

POWERS → **P05-05** · **P05-06**

storage_state

Log in once, reuse everywhere.

When: most tests need a logged-in user.

```
context.storage_state(path="customer.json")
browser.new_context(storage_state="customer.json")
```

POWERS → **P05-06**

page.route

Fake, block or modify network calls.

When: rare states: empty lists, 500s, slow responses.

```
page.route("**/products*", lambda r: r.fulfill(json=EMPTY_PAGE))
```

POWERS → **P05-06**

Evidence on failure

Trace, screenshot, video — only when red.

When: always on in CI.

```
# pytest --tracing retain-on-failure --screenshot only-on-failure
```

POWERS → **P05-07**

P05-01 · ZERO · 3H

First Playwright tests on SauceDemo

App — SauceDemo

Deliverable — `tests/ui/saucedemo/test_login.py`

The task

1. `uv run playwright install --with-deps chromium firefox webkit` (drop `--with-deps` on macOS).
2. Explore with `uv run playwright codegen https://www.saucedemo.com` and read the generated code.
3. Write tests: `standard_user` logs in → URL ends with `/inventory.html` and the title shows **Products**; `locked_out_user` → the error message is visible; empty username → "Username is required".
4. Run them headed, with `--slowmo 500`, and with `--browser firefox`.
5. Set `base_url` so tests call `page.goto("/")`.

`done when` Tick these off

- [ ] 3 tests pass on Chromium and Firefox
- [ ] You can explain browser vs context vs page, and why `expect` beats `assert` for UI

📘 User guide

`concepts` What you need to know

- **Browser → Context → Page.** A *browser* is one launched engine (Chromium, Firefox or WebKit). A *context* is an isolated incognito-like profile (cookies, storage), cheap to create. A *page* is a tab.
- **pytest-playwright fixtures:** `page` (fresh context and page **per test**), `context`, `browser` (session), `browser_name`, `base_url`, plus CLI flags `--browser`, `--headed`, `--slowmo`, `--base-url`.
- **Locator:** a *lazy* description of how to find an element (`page.get_by_role("button", name="Login")`). It's resolved at action time.
- **Auto-waiting:** before acting, Playwright waits until the element is attached, visible, stable, enabled and able to receive events.
- **Web-first assertions:** `expect(locator).to_have_text(...)` *retries* until it passes or times out (5 s by default).

`deep dive` Going deeper

Why Playwright? It drives Chromium, Firefox and WebKit through one API, auto-waits on every action, isolates tests with cheap contexts, intercepts the network, records traces, and

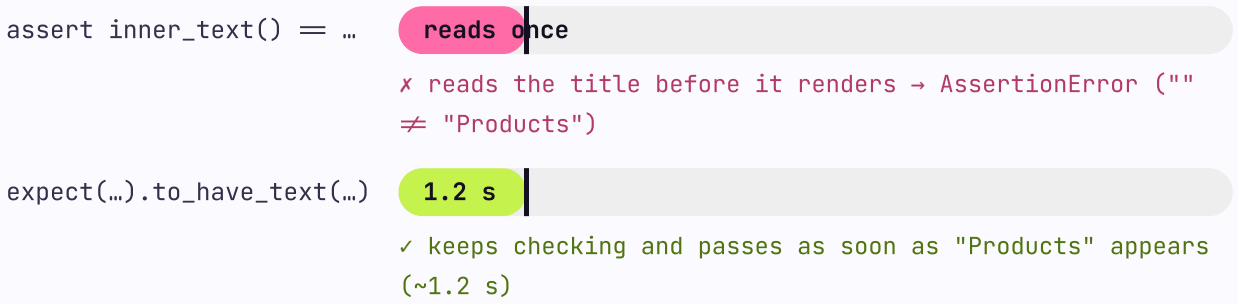
runs headless fast. Compared with Selenium/WebDriver: fewer explicit waits, built-in tracing and network control, and multi-tab/iframe handling without switching.

`expect` **vs** `assert`**.** `assert page.locator("h1").inner_text() == "Products"` reads the value *once*: if the page is still rendering, it fails. `expect(page.locator(".title")).to_have_text("Products")` polls until true or timeout. **Always use** `expect` **for UI state.** Use plain `assert` for already-computed values.

Test isolation for free. pytest-playwright gives each test a new context, so there's no cookie or localStorage leakage between tests. That's why UI tests can run in parallel safely (P07).

Headless vs headed: same engine, but headless has no visible window, is faster, and is the default in CI. Rare differences (viewport size, fonts, GPU) are why CI runs in a pinned Docker image (P08).

INTERACTIVE — A SNAPSHOT VS A RETRYING ASSERTION



The vertical line marks when the page shows “Products”. A snapshot races the render; `expect` waits for the condition.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: valid login.**

tests/ui/saucedemo/test_login.py

```python
# tests/ui/saucedemo/test_login.py
import re
from playwright.sync_api import Page, expect

def test_standard_user_can_login(page: Page):
    page.goto("https://www.saucedemo.com/")
    page.get_by_placeholder("Username").fill("standard_user")
    page.get_by_placeholder("Password").fill("secret_sauce")
    page.get_by_role("button", name="Login").click()
    expect(page).to_have_url(re.compile(r".*/inventory\.html"))
    expect(page.locator(".title")).to_have_text("Products")
```

SAY THIS

The locators describe how to find each element; `expect` keeps retrying until the page really shows Products.

LEVEL 2 — IMPRESS THEM

🔵 Medium: base URL and negative cases.

pyproject.toml

```toml
# pyproject.toml → [tool.pytest.ini_options]
addopts = "-ra --strict-markers --base-url https://www.saucedemo.com"
```

```python
import pytest
from playwright.sync_api import Page, expect

@pytest.mark.parametrize("username, password, error", [
    ("locked_out_user", "secret_sauce", "Epic sadface: Sorry, this user has been locked
out."),
    ("", "secret_sauce", "Epic sadface: Username is required"),
    ("standard_user", "", "Epic sadface: Password is required"),
    ("standard_user", "wrong", "Epic sadface: Username and password do not match any user
in this service"),
], ids=["locked", "no-username", "no-password", "wrong-password"])
def test_login_errors(page: Page, username, password, error):
    page.goto("/")
    page.get_by_placeholder("Username").fill(username)
    page.get_by_placeholder("Password").fill(password)
    page.get_by_role("button", name="Login").click()
    expect(page.locator("[data-test='error']")).to_have_text(error)
```

HIRE SIGNAL

`base_url` keeps URLs out of tests, and one parametrized test covers every login error with readable ids.

LEVEL 3 — HERO MODE

🟣 High/Complex: context configuration and a reusable logged-in fixture.

tests/ui/saucedemo/conftest.py

```python
# tests/ui/saucedemo/conftest.py
import pytest
from playwright.sync_api import Page, expect

@pytest.fixture(scope="session")
def browser_context_args(browser_context_args):
    """Override pytest-playwright's defaults for every context in this folder."""
    return {**browser_context_args, "viewport": {"width": 1366, "height": 768},
            "locale": "en-US", "timezone_id": "Europe/Amsterdam"}

@pytest.fixture(scope="session", autouse=True)
def use_data_test_attribute(playwright):
    playwright.selectors.set_test_id_attribute("data-test")   # get_by_test_id → [data-test=...]

@pytest.fixture
def logged_in_page(page: Page) -> Page:
    page.goto("/")
    page.get_by_test_id("username").fill("standard_user")
    page.get_by_test_id("password").fill("secret_sauce")
    page.get_by_test_id("login-button").click()
    expect(page.locator(".title")).to_have_text("Products")
    return page

def test_inventory_shows_six_products(logged_in_page: Page):
    expect(logged_in_page.locator(".inventory_item")).to_have_count(6)
```

WHY IT'S HERO-LEVEL

`browser_context_args` fixes viewport, locale and timezone for every context, and `logged_in_page` gives each test a fresh, logged-in page.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Playwright vs Selenium: why would you choose Playwright?**

Built-in auto-waiting and web-first assertions (less flakiness, no explicit wait code), one API for Chromium, Firefox and WebKit, cheap isolated browser contexts, network interception/mocking, trace viewer, codegen, and native handling of tabs, frames and downloads. Selenium's strengths are its broader language and grid ecosystem, real-browser vendor drivers, and legacy support.

Z **Browser vs context vs page?**

The browser is the launched engine process. A context is an isolated session inside it (its own cookies and storage, like an incognito profile), cheap to create. A page is a tab within a context. Tests typically share a browser and get a fresh context per test for isolation.

H **How does Playwright's auto-waiting work?**

Before an action, Playwright runs actionability checks on the resolved element (attached to the DOM, visible, stable/not animating, receives events/not covered, enabled, editable for inputs) and retries until they pass or the action times out. `expect` assertions similarly re-query and retry until the condition holds or times out.

H **Why is `expect(locator).to_have_text()` better than `assert locator.inner_text() == ...`?**

`inner_text()` takes a one-time snapshot, so it races with rendering and fails intermittently. `expect` retries until the condition is true (up to a timeout), matching how UIs actually update, and produces a descriptive failure. Retrying assertions are the main tool against UI flakiness.

Z **What does codegen give you, and what are its limits?**

It records your actions into code and suggests locators, which is great for discovering selectors and bootstrapping. Limits: no structure (no page objects or fixtures), weak assertions, and sometimes brittle locators. Treat the output as a draft to refactor.

🤖 AI ASSIST

Paste codegen output into AI and ask it to *refactor into a clean pytest test with* `expect` *assertions*, then compare with your own refactor.

P05-02 · 🟢 ZERO · 3H

Locator strategy and data-driven login

App — SauceDemo

Deliverable — `tests/ui/saucedemo/test_login_matrix.py`, `docs/locator-strategy.md`

📋 The task

1. Drive login tests from `saucedemo_users.csv` (P02-03): users that can log in reach Products, and the rest see their expected error.
2. For the **Login button** and the **Backpack's "Add to cart" button**, write 6 different locators (role, test id, placeholder/text, CSS id, CSS class, XPath). Rank them in `docs/locator-strategy.md` with reasons.
3. Add a specific product to the cart **by name** using `filter(has_text=...)` and verify that the cart badge shows `1`.
4. Trigger a strict-mode violation deliberately (a locator matching 6 elements) and read the error.

done when Tick these off

- [ ] The CSV-driven test covers every SauceDemo user
- [ ] Your locator doc states your team's priority order

📘 User guide

concepts What you need to know

- **User-facing locators:** `get_by_role`, `get_by_label`, `get_by_placeholder`, `get_by_text`, `get_by_alt_text`, `get_by_title`.
- **Test ids:** `get_by_test_id` (default attribute `data-testid`, configurable, e.g. SauceDemo's `data-test`).
- **CSS/XPath:** `page.locator("css=...")`, `page.locator("xpath=...")`.
- **Chaining and filtering:** `locator.filter(has_text=..., has=...)`, `.first`, `.nth(i)`, `locator.locator(...)`, `locator.get_by_role(...)`.
- **Strictness:** actions on a locator matching **more than one** element throw an error rather than silently clicking the first.

deep dive Going deeper

Priority order (and why):

1. **Role + accessible name:** `get_by_role("button", name="Login")`. Mirrors how users and assistive tech see the page, and survives CSS/markup refactors.
2. **Label / placeholder / text:** also user-facing, but text changes with copy edits and localisation.
3. **Test id:** `get_by_test_id("login-button")`. Stable by contract with developers, invisible to users, and ideal when roles are ambiguous.
4. **CSS** (stable ids or classes): acceptable, but tied to implementation.
5. **XPath:** a last resort, brittle with structure changes. Positional XPath (`//div[3]/button`) is the classic flaky locator.

Locator vs ElementHandle. A locator is re-resolved on every use, so it survives re-renders. An `ElementHandle` points to one specific DOM node that may go stale after React re-renders. Prefer locators always.

Scoping beats global search. "The Add-to-cart button *inside the Backpack card*" is `cards.filter(has_text="Sauce Labs Backpack").get_by_role("button", name="Add to cart")`. It's precise, readable and immune to the other five buttons.

Shadow DOM: Playwright CSS and role locators pierce *open* shadow roots automatically. XPath doesn't.

INTERACTIVE — LOCATOR PLAYGROUND ON A TINY SAUCEDEMO-LIKE PAGE

role
```
page.get_by_role("button", name="Login")
```

test id
```
page.get_by_test_id("login-button")
```

placeholder
```
page.get_by_placeholder("Username")
```

CSS id
```
page.locator("#login-button")
```

CSS class
```
page.locator("input.submit-button")
```

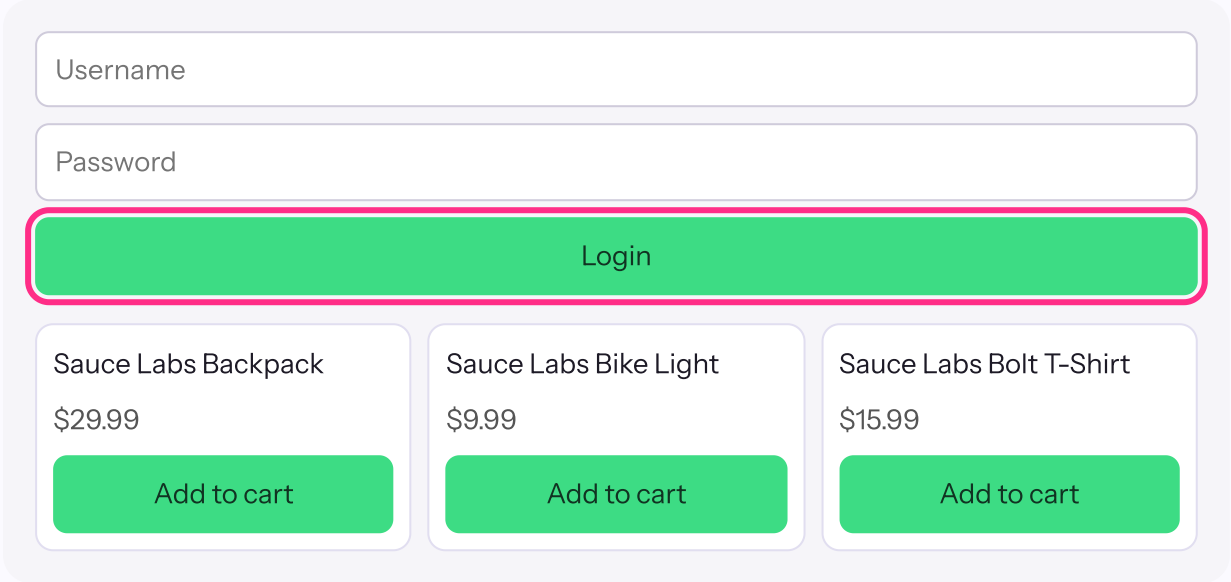
XPath
```
page.locator("xpath=//input[@type='submit']")
```

positional XPath
```
page.locator("xpath=/html/body/div/div/form/input[3]")
```

all Add buttons
```
page.get_by_role("button", name="Add to cart")
```

scoped
```
page.locator(".inventory_item").filter(has_text="Sauce Labs
Backpack").get_by_role("button", name="Add to cart")
```

.first
```
page.get_by_role("button", name="Add to cart").first
```




```
page.get_by_role("button", name="Login")
matches: 1 · ✓ .click() acts on exactly one element
```

```
Today every Login strategy finds exactly one element: role ✓ · test id ✓ ·
placeholder ✓ · CSS id ✓ · CSS class ✓ · XPath ✓ · positional XPath ✓
```

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: user-facing locators and one scoped action.

```python
from playwright.sync_api import Page, expect

def test_add_backpack_to_cart(logged_in_page: Page):
    card = logged_in_page.locator(".inventory_item").filter(has_text="Sauce Labs Backpack")
    card.get_by_role("button", name="Add to cart").click()
    expect(card.get_by_role("button", name="Remove")).to_be_visible()
    expect(logged_in_page.locator(".shopping_cart_badge")).to_have_text("1")
```

SAY THIS

Scope first: find the Backpack's card, then the button inside it — immune to the other five buttons.

LEVEL 2 — IMPRESS THEM

🔵 Medium: data-driven login from CSV.

```python
import re
import pytest
from playwright.sync_api import Page, expect
from python_basics.data_io import load_csv

USERS = load_csv("saucedemo_users.csv")

@pytest.mark.parametrize("user", USERS, ids=lambda u: u["username"])
def test_login_outcome_per_user(page: Page, user):
    page.goto("/")
    page.get_by_placeholder("Username").fill(user["username"])
    page.get_by_placeholder("Password").fill(user["password"])
    page.get_by_role("button", name="Login").click()
    if user["can_login"]:
        expect(page).to_have_url(re.compile(r"/inventory\.html$"), timeout=15_000)   # performance_glitch_user is slow
    else:
        expect(page.locator("[data-test='error']")).to_contain_text(user["expected_error"])
```

NOTE

The `if` is acceptable here because the *data* defines two outcomes. If the branches grow, split them into two tests.

HIRE SIGNAL

The CSV defines two outcomes, so one data-driven test with a single `if` is acceptable; more branches would mean two tests.

LEVEL 3 — HERO MODE

🟣 **High/Complex: a locator comparison, a strict-mode lesson and multi-item assertions.**

```python
from playwright.sync_api import Page, expect, Error
import pytest

LOGIN_BUTTON_LOCATORS = {
    "role":      lambda p: p.get_by_role("button", name="Login"),
    "test_id":   lambda p: p.get_by_test_id("login-button"),          # data-test via
set_test_id_attribute
    "css_id":    lambda p: p.locator("#login-button"),
    "css_class": lambda p: p.locator("input.submit-button"),
    "xpath":     lambda p: p.locator("xpath=//input[@type='submit']"),
}

@pytest.mark.parametrize("strategy", LOGIN_BUTTON_LOCATORS)
def test_every_strategy_finds_exactly_one(page: Page, strategy):
    page.goto("/")
    expect(LOGIN_BUTTON_LOCATORS[strategy](page)).to_have_count(1)

def test_strict_mode_protects_from_ambiguous_clicks(logged_in_page: Page):
    all_buttons = logged_in_page.get_by_role("button", name="Add to cart")
    expect(all_buttons).to_have_count(6)
    with pytest.raises(Error, match="strict mode violation"):
        all_buttons.click(timeout=2_000)

def test_all_product_names_are_unique(logged_in_page: Page):
    names = logged_in_page.locator(".inventory_item_name").all_inner_texts()
    assert len(names) == len(set(names)) == 6, names
```

WHY IT'S HERO-LEVEL

Every strategy finds exactly one Login button today. Strict mode refuses to guess between six. Try the redesign toggle above to see which strategies survive.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **What locator strategy do you follow, in priority order?**

Role with accessible name first, then label/placeholder/text, then dedicated test ids, then stable CSS, and XPath only as a last resort. User-facing locators are resilient and double as accessibility checks. Test ids are a stable contract with developers.

H **What is strict mode in Playwright?**

Actions on a locator that matches multiple elements throw a "strict mode violation" instead of acting on the first match. It prevents silent wrong-element clicks. Fix it by scoping (`filter` , chaining inside a container) or, if intended, `.first` / `.nth()` .

H **Locator vs ElementHandle?**

A Locator is a lazy query re-resolved each time it's used, with auto-waiting and strictness. It survives re-renders. An ElementHandle is a reference to a specific DOM node that can go stale when the framework re-renders. Playwright recommends locators for almost everything.

H **How do you handle elements with dynamic IDs like `input-8472` ?**

Don't use the dynamic part. Use role and accessible name, labels, stable test ids (ask developers to add them), or partial attribute matches (`[id^="input-"]`) scoped inside a stable container. Relational filters (`has_text` , `has=`) are usually cleaner than clever CSS.

SH **How do you get developers to support testability (test ids, accessible names)?**

Make it part of the Definition of Done and the component library (test ids built into shared components), show the cost of brittle locators in flake metrics, add a lint rule or review checklist item, and frame accessible names as an accessibility win, not just a testing need. Pair on the first few components.

🤖 AI ASSIST

Paste a snippet of page HTML and ask AI for the best user-facing locator, then verify it in the Playwright Inspector (`page.pause()`).

P05-03 · 🔵 HERO · 4H

Page Object Model: SauceDemo purchase end-to-end

App — SauceDemo

Deliverable — `framework/pages/saucedemo/` , `tests/ui/saucedemo/test_checkout.py`

📋 The task

1. Page objects: `LoginPage` , `InventoryPage` , `CartPage` , `CheckoutInfoPage` , `CheckoutOverviewPage` , `CompletePage` , plus a `Header` component (cart badge, menu, logout).
2. Methods express intent and **return the next page object** (`login() → InventoryPage`).
3. E2E: buy 2 items, then verify that item total, tax and total **equal** `calculate_totals()` **from P02-02**.
4. Sorting: for each of the 4 sort modes, assert the order with `is_sorted()` from P02-01.
5. Provide page objects to tests via fixtures.

done when **Tick these off**

- [ ] Tests contain **no raw locators**, only page-object calls and `expect`
- [ ] The checkout totals test proves the UI maths matches your Python function

📘 User guide

`concepts` What you need to know

- **Page Object:** a class representing a page, exposing *what users can do* (`add_to_cart(name)`) and hiding *how* (locators).
- **Component object:** a reusable part (header, modal, table) composed into pages.
- **Fluent navigation:** methods return the page object you land on.

`deep dive` Going deeper

POM rules of thumb:

1. **Locators live in one place:** the page object (as properties or attributes).
2. **Methods model user intent**, not clicks: `checkout_with(first, last, zip)` rather than `click_continue()` everywhere.
3. **Assertions live in tests.** Page objects may expose *state* (`badge_count()`, `prices()`) and wait for "page loaded". The team decides whether pages may contain `expect`s for load checks; be consistent.
4. **Return page objects** to make flows discoverable (the IDE autocompletes the next step).
5. **No god objects:** split big pages into components.

Alternatives: the *Screenplay pattern* (actors, tasks and questions) scales better for huge suites with many roles. Some teams use *app actions* (setting state via API instead of UI steps). Our hybrid tests in P05-06 do exactly that.

Parsing UI numbers: convert `"Item total: $29.99"` to `Decimal("29.99")` with one helper, reused everywhere. Money assertions stay exact.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: LoginPage.

framework/pages/saucedemo/login_page.py

```python
# framework/pages/saucedemo/login_page.py
from playwright.sync_api import Page, expect

class LoginPage:
    def __init__(self, page: Page) -> None:
        self.page = page
        self.username = page.get_by_placeholder("Username")
        self.password = page.get_by_placeholder("Password")
        self.submit = page.get_by_role("button", name="Login")
        self.error = page.locator("[data-test='error']")

    def open(self) -> "LoginPage":
        self.page.goto("/")
        return self

    def login(self, user: str, password: str = "secret_sauce") -> "InventoryPage":
        from framework.pages.saucedemo.inventory_page import InventoryPage
        self.username.fill(user)
        self.password.fill(password)
        self.submit.click()
        return InventoryPage(self.page).wait_loaded()
```

SAY THIS

Locators live in the page object, and `login()` returns the next page, so the flow autocompletes in the IDE.

LEVEL 2 — IMPRESS THEM

🔵 Medium: InventoryPage with a Header component and sorting.

framework/pages/saucedemo/inventory_page.py

```python
# framework/pages/saucedemo/inventory_page.py
from decimal import Decimal
from playwright.sync_api import Page, expect

class Header:
    def __init__(self, page: Page) -> None:
        self.page = page
        self.badge = page.locator(".shopping_cart_badge")
        self.cart_link = page.locator(".shopping_cart_link")

    def cart_count(self) -> int:
        return int(self.badge.inner_text()) if self.badge.count() else 0

    def open_cart(self) -> "CartPage":
        from framework.pages.saucedemo.cart_page import CartPage
        self.cart_link.click()
        return CartPage(self.page)

class InventoryPage:
    def __init__(self, page: Page) -> None:
        self.page = page
        self.header = Header(page)
        self.title = page.locator(".title")
        self.items = page.locator(".inventory_item")
        self.sort_select = page.locator(".product_sort_container")

    def wait_loaded(self) -> "InventoryPage":
        expect(self.title).to_have_text("Products")
        return self

    def add_to_cart(self, product: str) -> "InventoryPage":
        self.items.filter(has_text=product).get_by_role("button", name="Add to
cart").click()
        return self

    def sort_by(self, mode: str) -> "InventoryPage":          # az | za | lohi | hilo
        self.sort_select.select_option(mode)
        return self

    def names(self) -> list[str]:
        return self.page.locator(".inventory_item_name").all_inner_texts()

    def prices(self) -> list[Decimal]:
        return [Decimal(t.replace("$", "")) for t in
self.page.locator(".inventory_item_price").all_inner_texts()]
```

HIRE SIGNAL

`Header` is a component composed into pages, and `prices()` returns Decimals so the assertions stay exact.

LEVEL 3 — HERO MODE

🟣 **High/Complex: the full purchase with totals verified against your Python model.**

framework/pages/saucedemo/checkout_pages.py

```
# framework/pages/saucedemo/checkout_pages.py
import re
from decimal import Decimal
from playwright.sync_api import Page, expect

def money(text: str) -> Decimal:
    return Decimal(re.search(r"\$([\d.]+)", text).group(1))

class CheckoutInfoPage:
    def __init__(self, page: Page) -> None:
        self.page = page

    def continue_with(self, first: str, last: str, postal: str) -> "CheckoutOverviewPage":
        self.page.get_by_placeholder("First Name").fill(first)
        self.page.get_by_placeholder("Last Name").fill(last)
        self.page.get_by_placeholder("Zip/Postal Code").fill(postal)
        self.page.get_by_role("button", name="Continue").click()
        return CheckoutOverviewPage(self.page)

class CheckoutOverviewPage:
    def __init__(self, page: Page) -> None:
        self.page = page

    def totals(self) -> dict[str, Decimal]:
        expect(self.page.locator(".summary_total_label")).to_be_visible()
        return {
            "item_total": money(self.page.locator(".summary_subtotal_label").inner_text()),
            "tax": money(self.page.locator(".summary_tax_label").inner_text()),
            "total": money(self.page.locator(".summary_total_label").inner_text()),
        }

    def finish(self) -> None:
        self.page.get_by_role("button", name="Finish").click()
        expect(self.page.locator(".complete-header")).to_have_text("Thank you for your
order!")
```

tests/ui/saucedemo/test_checkout.py

```python
# tests/ui/saucedemo/test_checkout.py
import pytest
from decimal import Decimal
from playwright.sync_api import expect
from python_basics.checkout import calculate_totals
from python_basics.products import is_sorted
from framework.pages.saucedemo.login_page import LoginPage

@pytest.fixture
def inventory(page):
    return LoginPage(page).open().login("standard_user")

def test_checkout_totals_match_business_rules(inventory):
    inventory.add_to_cart("Sauce Labs Backpack").add_to_cart("Sauce Labs Bike Light")
    expect(inventory.header.badge).to_have_text("2")      # retrying check: cart_count()
would race the re-render
    overview = inventory.header.open_cart().checkout().continue_with("Ada", "Lovelace",
"1234AB")
    assert overview.totals() == calculate_totals([Decimal("29.99"), Decimal("9.99")])
    overview.finish()

@pytest.mark.parametrize("mode, key, reverse", [
    ("az", "names", False), ("za", "names", True), ("lohi", "prices", False), ("hilo",
"prices", True)])
def test_sorting(inventory, mode, key, reverse):
    inventory.sort_by(mode)
    values = getattr(inventory, key)()
    assert is_sorted(values, reverse=reverse), f"{mode}: {values}"
```

NOTE

You still need `CartPage.checkout() → CheckoutInfoPage`. Write it yourself.

WHY IT'S HERO-LEVEL

The UI's totals are compared with `calculate_totals()` from P02 — the test checks the business rule, not a magic number.

try it The money() helper, checked against P02 + ADDED

This part is plain Python, so it runs here. Look at the last line: the regex stops at the comma, the same bug as P03-02's `parse_price`. SauceDemo totals never reach $1,000, but a reusable helper shouldn't depend on that.

```python
# + added: the P05-03 money() helper, checked against P02's calculate_totals()
import re
from decimal import Decimal

from python_basics.checkout import calculate_totals


def money(text: str) -> Decimal:                          # exactly as in checkout_pages.py
    return Decimal(re.search(r"\$([\d.]+)", text).group(1))


overview = {"item_total": "Item total: $39.98", "tax": "Tax: $3.20", "total": "Total:
$43.18"}  # Backpack + Bike Light
from_ui = {key: money(text) for key, text in overview.items()}
assert from_ui == calculate_totals([Decimal("29.99"), Decimal("9.99")])
print("UI totals match the Python model:", from_ui)

print("but with a thousands separator:", money("Total: $1,299.00"))   # the P03-02
parse_price bug again
```

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z What is the Page Object Model and why use it?

A design pattern where each page (or component) is a class that encapsulates its locators and exposes user-level actions. Benefits: locator changes are fixed in one place, tests read like user flows, and code is reused. Costs: boilerplate, and poorly designed page objects become huge and leak details.

H Should page objects contain assertions?

Mostly no. Tests should own verification, so what's checked is visible in the test. Page objects expose state (texts, counts, values) and may include readiness waits (e.g. `wait_loaded()` with `expect` on a landmark). Consistency across the team matters more than the choice.

H **What are component objects and when do you use them?**

Objects for reusable UI parts (header, nav, modal, table, date picker) that appear on multiple pages. Pages compose them (`inventory.header.cart_count()`). They avoid duplicated locators across pages and model the UI the way modern component-based front ends are built.

SH **POM vs Screenplay: when would you switch?**

POM is simple and great up to medium suites. Screenplay (actors with abilities performing tasks and asking questions) scales better with many roles, complex multi-actor flows and cross-channel steps (API + UI + email), because behaviour is composed from small reusable tasks rather than growing page classes. Switch when page classes become god objects or when flows span many pages and roles.

SH **How would you structure a UI framework used by 10 product teams?**

A shared core package (config, browser and context fixtures, auth-state helpers, reporting hooks, base components like tables and modals, API clients for setup) versioned and released separately, with team-owned page objects and tests in product repos. Add conventions (locator policy, no hard waits, data ownership), templates and linting, CODEOWNERS on the core, and dashboards for flake and duration per team.

🤖 AI ASSIST

Ask AI to generate the `CartPage` from the page HTML, then check it follows *your* conventions (return types, no assertions, locator style). Consider encoding those conventions as a skill in P12-03.

P05-04 · 🔵 HERO · 5H

Rescue a legacy suite

App — SauceDemo (`legacy-suite/playwright/`)

Deliverable — `docs/legacy-rescue.md` , `tests/ui/saucedemo/test_legacy_rescued.py`

📋 The task

> WHY THIS MATTERS
>
> new QEs rarely start with a blank repo. On day 1 you usually inherit a suite someone else wrote, and the team is used to "just rerun it". This task is that day, on purpose.

1. Read `legacy-suite/README.md` . **Measure before you judge:** run the suite in file order with `--durations=0` , then run single tests on their own, then run two tests in reverse order (🟢 example). Record the runtime, the time spent sleeping and the result in each mode.
2. **Smell inventory:** in `docs/legacy-rescue.md` , list every smell you find with its location, a category (waits, locators, isolation, assertions, data, structure, process) and the risk it creates. Aim for at least 15.
3. **Prove the two worst ones:** show a test that *can't fail* (change what it checks so it should fail, and watch it stay green), and show a test that fails *only because of order*.
4. **Rescue the intent:** rewrite what each legacy test was *trying* to protect as `tests/ui/saucedemo/test_legacy_rescued.py` , using your P05-03 page objects. Make the tests independent, log in through a fixture, use `expect` instead of sleeps, use user-facing locators, and write correct assertions. The commented-out locked-out test is already covered by your P05-02 login matrix, so link to it rather than duplicate it.
5. **Measure after:** runtime, each test run on its own, and 10 runs in random order (🟣 example).
6. Add a **"What I'd tell the team"** section: which smells to fix first and why, and how to stop them coming back (lint rules, a review checklist, a `CLAUDE.md` rule).

`done when` Tick these off

- [ ] `docs/legacy-rescue.md` has the baseline, at least 15 smells with their risks, and before/after numbers
- [ ] Every rescued test passes on its own, in any order, 10 runs in a row
- [ ] The rescued sort test fails if you flip its expected order (proof that it can fail)

- [ ] Nothing in `legacy-suite/` was changed: it stays as your "before"

User guide

concepts What you need to know

- **Test smell:** a pattern in test code that signals a deeper problem, such as tests that are slow, flaky, hard to read or unable to fail. The classic catalogue is Gerard Meszaros's *xUnit Test Patterns*: fragile test, erratic test, obscure test, slow test, assertion roulette, interacting tests.
- **Brownfield vs greenfield:** improving an existing, in-use codebase vs starting fresh. Most real QE work is brownfield.
- **Characterisation test:** a test that records what the system does *today*, so you can refactor safely before deciding what it *should* do.
- **Strangler approach:** replace an old system piece by piece while it keeps running: new tests are written the new way, and old ones are migrated by risk and deleted when replaced.
- **Flake rate:** failed runs ÷ total runs for the same code. Anything above zero on unchanged code needs an explanation.

deep dive Going deeper

Order of work when you inherit a suite:

1. **Run it and measure** before forming opinions. Numbers survive disagreements; "this code is ugly" doesn't.
2. **Read for intent.** For each test, write one sentence: "this protects …". Tests with no clear intent are candidates for deletion, not rescue.
3. **Rank smells by risk, not by ugliness.** Most dangerous first: tests that *can't fail* (they give false confidence), then tests whose result depends on order or timing (they train people to ignore red), then slow ones (they get skipped), and cosmetic problems last.
4. **Fix in small, reviewable PRs** that keep behaviour, and measure after each one.
5. **Prevent regressions:** lint rules, a review checklist, instruction-file rules for AI assistants, and a CI gate on flake rate (P10-03).

Don't big-bang rewrite a suite people depend on. A rewrite that takes three months leaves the team with no regression safety net in the meantime and usually recreates half the old problems. Strangle it instead: every new test follows the new rules, legacy tests are migrated in risk order, and each one is deleted the day its replacement is green.

"Just rerun it" is a process smell. A team that reruns red builds until they're green has stopped trusting its tests, and that's the moment real bugs start slipping through. Part of the rescue is social: show the numbers, fix the worst offenders visibly, and agree a flaky-test policy (P10-03).

A finding in your own code: the legacy glitch test "works" because it sleeps 6 seconds. Your rescued version will probably fail at first, because `LoginPage.login()` waits for the products page with `expect`'s **default 5-second timeout**, and `performance_glitch_user` takes about 5.1 seconds. The fix is *not* a sleep: make the wait's timeout explicit and justified for that user (🟣 example).

INTERACTIVE — THE LEGACY SUITE'S SHARED PAGE, TEST BY TEST

test_1 PASSED logs in by absolute XPath, sleeps 3 s, sees the inventory

test_2 PASSED adds the first product, stores its name in the module-level DATA dict, badge shows 1

test_3 PASSED opens the cart and finds the name test_2 left in DATA

test_checkout PASSED checks out the one item, total == 32.39 (as a float!), sees "Thank you"

test_sort PASSED logs in again and sorts — and its wrong assertion is swallowed by except Exception: pass, so it can never fail

test_glitch PASSED logs in as performance_glitch_user and passes only because it sleeps 6 s

test_logout PASSED logs out and passes: it asserts nothing at all

shared page on: **login** | cart: **0** | DATA["item"] set: **yes** | time so far: **~24 s** | failed: **0**

✓ **test_logout**: logs out and passes: it asserts nothing at all · **0** of 7 failed in this order, about 24 s in total.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: measure the baseline.

```
uv run pytest legacy-suite/playwright -v --durations=0              # file order: green, and slow
grep -o "sleep([0-9.]*)" legacy-suite/playwright/test_shop.py       # add these up: that's wasted time
uv run pytest "legacy-suite/playwright/test_shop.py::test_3" -v     # alone: does it still pass?
uv run pytest "legacy-suite/playwright/test_shop.py::test_3" "legacy-suite/playwright/test_shop.py::test_2" -v   # reversed
```

```
## Smell inventory (first three rows, find the rest yourself)
| # | Where | Smell | Category | Risk |
|---|---|---|---|---|
| 1 | conftest.py (module level) | Browser and page created at import time and shared by every test | Isolation | Results depend on order; one failure cascades into every later test |
| 2 | test_shop.py `test_1` | `time.sleep(3)` after login | Waits | Always slow; flaky whenever the site takes more than 3 s |
| 3 | test_shop.py `test_sort` | `except Exception: pass` around the assertion | Assertions | The test can never fail, whatever the app does |
```

SAY THIS

Measure before judging: total runtime, time spent asleep, and which tests pass on their own or in reverse order.

LEVEL 2 — IMPRESS THEM

🔵 Medium: the rescued tests.

tests/ui/saucedemo/test_legacy_rescued.py

```python
# tests/ui/saucedemo/test_legacy_rescued.py
from decimal import Decimal

import pytest
from playwright.sync_api import Page, expect

from framework.pages.saucedemo.inventory_page import InventoryPage
from framework.pages.saucedemo.login_page import LoginPage
from python_basics.checkout import calculate_totals
from python_basics.products import is_sorted


@pytest.fixture
def inventory(page: Page) -> InventoryPage:
    return LoginPage(page).open().login("standard_user")  # every test gets its own fresh, logged-in page


def test_adding_a_product_updates_the_cart_badge(inventory: InventoryPage) -> None:
    inventory.add_to_cart("Sauce Labs Backpack")
    expect(inventory.header.badge).to_have_text("1")


def test_price_low_to_high_sorts_ascending(inventory: InventoryPage) -> None:
    inventory.sort_by("lohi")
    prices = inventory.prices()
    # The legacy test asserted DEscending order, then swallowed the failure.
    assert is_sorted(prices), f"not ascending: {prices}"


def test_checkout_totals_follow_the_pricing_rules(inventory: InventoryPage) -> None:
    overview = inventory.add_to_cart("Sauce Labs Backpack").header.open_cart().checkout().continue_with(
        "Test", "User", "12345"
    )
    assert overview.totals() == calculate_totals([Decimal("29.99")])  # from the rules, not a magic 32.39
    overview.finish()
```

Add the cart-contents and logout tests yourself; they need your `CartPage` and a small `menu` component.

> HIRE SIGNAL
>
> *Each rescued test logs in through a fixture, waits with `expect` instead of sleeping, and can fail — the sort test asserts the right order now.*

LEVEL 3 — HERO MODE

🟣 High/Complex: a justified timeout, and proof of stability.

framework/pages/saucedemo/login_page.py

```
# framework/pages/saucedemo/login_page.py: give the readiness wait an explicit, overridable
timeout
def login(self, user: str, password: str = "secret_sauce", timeout_ms: float = 5_000) →
"InventoryPage":
    from framework.pages.saucedemo.inventory_page import InventoryPage

    self.username.fill(user)
    self.password.fill(password)
    self.submit.click()
    return InventoryPage(self.page).wait_loaded(timeout_ms)


# tests/ui/saucedemo/test_legacy_rescued.py
SLOW_LOGIN_MS = 10_000  # performance_glitch_user takes ~5.1 s by design: allow 2x, don't
sleep


def test_slow_user_still_reaches_products(page: Page) → None:
    inventory = LoginPage(page).open().login("performance_glitch_user",
timeout_ms=SLOW_LOGIN_MS)
    expect(inventory.title).to_have_text("Products")
```

(`wait_loaded(timeout_ms)` passes the value on: `expect(self.title).to_have_text("Products", timeout=timeout_ms)`. Adjust the attribute names to match your page object.)

```
# 10 runs in random order: every line should say "pass"
for i in $(seq 10); do
  uv run --with pytest-randomly python -m pytest tests/ui/saucedemo/test_legacy_rescued.py
-q >/dev/null && echo pass || echo FAIL
done | sort | uniq -c
```

METRIC	BEFORE (LEGACY)	AFTER (RESCUED)
Runtime	about 25 s	measure it
Time spent in `sleep`	add up the sleeps	0 s
Tests that pass on their own	count them	all
Tests that can fail when the app is wrong	count them	all (prove it)

KEY **Smell answer key (open only after your own inventory)**

1. Browser, context and page created at import time in `conftest.py` (global state, no isolation, no clean teardown per test)
2. One page shared by every test, so the cart and login state leak between tests
3. Tests depend on each other's order (`test_2` needs `test_1` 's login, `test_3` needs `test_2` 's cart, `test_checkout` needs `test_3` , `test_logout` needs a logged-in page)
4. A module-level `DATA` dict passes values between tests
5. `from conftest import …` : importing from conftest is an anti-pattern (fixtures are injected, not imported)
6. Hard waits everywhere (`time.sleep` : around 24 s of the ~25 s runtime)
7. A sleep "tuned" to one machine (`time.sleep(6) # 6s is plenty`)
8. Absolute XPaths copied from DevTools (break on any layout change)
9. Non-retrying checks (`is_visible` , `inner_text` + `assert`) instead of `expect`
10. `except Exception: pass` swallows the sort assertion, so the test can't fail
11. The sort assertion is wrong anyway (low→high asserted as descending), hidden by #10
12. Money as `float`
13. A magic expected total (`32.39`) instead of one derived from price and tax rules
14. URL and credentials hard-coded in the tests (no config)
15. Copy-pasted login steps in several tests (no fixture or page object)
16. Meaningless test names (`test_1` , `test_2` , `test_3`)
17. `print` debugging left in
18. A test with no assertion (`test_logout`)
19. Assertions against the whole HTML (`in driver.content()`), which can pass for the wrong reason
20. Commented-out test: lost coverage that nobody will notice
21. Process smells in the handover note: "just rerun it" and "don't touch conftest.py"

WHY IT'S HERO-LEVEL

A known-slow user gets an explicit, justified timeout on that one wait — not a sleep — and ten random-order runs prove the suite is stable.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **What is a test smell? Give three examples.**

A pattern in test code that signals a deeper problem even if the test passes today. Examples: hard-coded sleeps (slow and flaky), tests that depend on each other's order (results change with order or parallelism), and assertions that can't fail (a swallowed exception, or checking something always true). Others: brittle absolute XPaths, magic numbers, duplicated setup and meaningless test names.

Z **Why is `time.sleep` a problem in UI tests?**

A fixed wait is wrong in both directions: too short on a slow day (flaky failures) and too long on a fast one (wasted time on every run, multiplied by every test). It also hides the real condition you're waiting for. Wait for the condition itself instead: Playwright's auto-waiting actions and retrying `expect` assertions, with an explicit, justified timeout where something is genuinely slow.

H **You inherit a slow, flaky suite that the team reruns until it's green. What do you do in your first month?**

Measure first: runtime, flake rate per test, and which failures are real. Read the tests for intent and rank the problems by risk: tests that can't fail, then order- and timing-dependent ones, then slow ones. Fix the worst offenders in small PRs and show before/after numbers. Quarantine known-flaky tests with owners and deadlines rather than silently retrying them. Agree a flaky-test policy and add guardrails (lint rules, a review checklist, CI flake tracking) so it doesn't drift back.

H **Refactor or rewrite a legacy test suite?**

Usually refactor incrementally, using the strangler approach: new tests follow the new standards, old ones are migrated in risk order, and each is deleted once its replacement is green. That keeps a safety net the whole time. A full rewrite is justified only when the suite gives no useful signal at all, or the tool is being replaced anyway, and even then it's done in slices, keeping both suites running until parity is proven.

SH **After a clean-up, how do you stop test smells coming back?**

Make the right way the easy way and the wrong way visible: shared fixtures and page objects that make good tests short, lint rules for the mechanical smells (ruff's pytest rules, bans on `time.sleep` in tests), a review checklist focused on "can this test fail?", rules in the AI instruction file, and CI metrics for duration and flake rate per test with alerts. Review the numbers regularly, because smells come back through deadlines, not ignorance.

🤖 AI ASSIST

- Ask AI to list smells in `legacy-suite/playwright/test_shop.py` **after** you've written your own inventory. Compare: what did it miss, and what did it flag that isn't really a problem?
- Let it draft one rescued test, then review it with the P00-06 checklist. Does it still use a sleep or a copied XPath anywhere?

P05-05 · 🔵 HERO · 4H

The Internet: tricky UI patterns

App — The Internet | **Deliverable** — `tests/ui/internet/` (≥12 tests)

📋 The task

Write one focused test for each pattern (base URL `https://the-internet.herokuapp.com`):

PAGE	WHAT TO PROVE
`/dynamic_loading/1` and `/2`	Waiting for hidden or late-rendered content, **with no sleeps**
`/checkboxes`, `/dropdown`	Form controls state
`/javascript_alerts`	Accept alert, dismiss confirm, type into prompt
`/windows`	New tab handling
`/nested_frames`	Nested iframe text
`/upload`, `/download`	File round trip
`/basic_auth`	HTTP credentials
`/hovers`	Hover reveals content
`/drag_and_drop`	Drag A onto B
`/tables`	Sort by column and verify the order
`/key_presses`	Keyboard input
`/shadowdom`	Text inside a shadow root

`done when` Tick these off

- [ ] All tests pass headless on Chromium, **and** on Firefox
- [ ] `grep -rn "wait_for_timeout\|time.sleep" tests/ui` returns nothing

📘 User guide

`concepts` What you need to know

- **Waiting toolkit (best first):** auto-waiting actions → `expect(...)` with a custom `timeout` → `locator.wait_for(state=...)` → `page.expect_response()` / `expect_request()` / `wait_for_url()` → *never* `wait_for_timeout`.

- **Dialogs:** `page.on("dialog", handler)` / `page.once(...)`. Unhandled dialogs are **auto-dismissed** by Playwright.
- **Frames:** `page.frame_locator("iframe#id").locator(...)`, which can be chained for nested frames.
- **New tabs/popups:** `with page.context.expect_page() as info: ...click()` then `info.value`.
- **Files:** `locator.set_input_files(path)`; `with page.expect_download() as info:` then `info.value.save_as(...)`.
- **HTTP auth:** `browser.new_context(http_credentials={...})`.

deep dive Going deeper

Why hard waits are banned. `wait_for_timeout(5000)` is always slow (wastes 5 s when the app needs 0.5 s) *and* flaky (fails when CI needs 6 s). Every sleep is a guess. Wait for a **condition**: an element's state, a URL, a network response. If the default 5 s `expect` timeout is too short for a known-slow operation, pass a larger `timeout=` **to that assertion only**.

Event-then-trigger order. For dialogs, popups and downloads, *register the expectation before the action that triggers it*: `with page.expect_download() as d: link.click()`. Registering after the click can miss the event. That's the most common bug in these tests.

Frames are separate documents. Locators don't cross into iframes automatically. Use `frame_locator` and chain it for nested frames. (Shadow DOM is different: Playwright pierces open shadow roots automatically.)

INTERACTIVE — REGISTER THE EXPECTATION FIRST, OR MISS THE EVENT

```
page.once("dialog", lambda d: d.accept("QE hero"))
```

```
click "Click for JS Prompt"
```

```
prompt opens → the handler accepts with "QE hero"
```

```
#result: "You entered: QE hero" ✓
```

The assertion passes.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: form controls and dynamic loading.

```python
from playwright.sync_api import Page, expect

BASE = "https://the-internet.herokuapp.com"

def test_checkbox_toggles(page: Page):
    page.goto(f"{BASE}/checkboxes")
    first = page.get_by_role("checkbox").nth(0)
    first.check()
    expect(first).to_be_checked()

def test_dropdown_selection(page: Page):
    page.goto(f"{BASE}/dropdown")
    page.locator("#dropdown").select_option("2")
    expect(page.locator("#dropdown")).to_have_value("2")

def test_late_rendered_element(page: Page):
    page.goto(f"{BASE}/dynamic_loading/2")
    page.get_by_role("button", name="Start").click()
    expect(page.locator("#finish")).to_have_text("Hello World!", timeout=10_000)   # known ~5s load
```

> SAY THIS
>
> *No sleeps: actions auto-wait, and only the assertion that needs it gets a longer timeout.*

LEVEL 2 — IMPRESS THEM

🔵 Medium: dialogs, new tabs and nested frames.

```python
def test_prompt_receives_text(page: Page):
    page.goto(f"{BASE}/javascript_alerts")
    page.once("dialog", lambda dialog: dialog.accept("QE hero"))       # register BEFORE
clicking
    page.get_by_role("button", name="Click for JS Prompt").click()
    expect(page.locator("#result")).to_have_text("You entered: QE hero")

def test_confirm_can_be_cancelled(page: Page):
    page.goto(f"{BASE}/javascript_alerts")
    page.once("dialog", lambda dialog: dialog.dismiss())
    page.get_by_role("button", name="Click for JS Confirm").click()
    expect(page.locator("#result")).to_have_text("You clicked: Cancel")

def test_new_window(page: Page):
    page.goto(f"{BASE}/windows")
    with page.context.expect_page() as new_page_info:
        page.get_by_role("link", name="Click Here").click()
    new_page = new_page_info.value
    expect(new_page.locator("h3")).to_have_text("New Window")

def test_nested_frames(page: Page):
    page.goto(f"{BASE}/nested_frames")
    middle = page.frame_locator("frame[name='frame-
top']").frame_locator("frame[name='frame-middle']")
    expect(middle.locator("#content")).to_have_text("MIDDLE")
```

HIRE SIGNAL

Register the dialog handler — or `expect_page` — before the click that triggers it; frames are reached with `frame_locator`.

LEVEL 3 — HERO MODE

🟣 **High/Complex: a file round trip, a sortable table and basic auth.**

```python
from pathlib import Path
import pytest
from playwright.sync_api import Browser, Page, expect
from python_basics.products import is_sorted

def test_download_then_upload_same_file(page: Page, tmp_path: Path):
    page.goto(f"{BASE}/download")
    with page.expect_download() as dl_info:
        page.locator("#content a").first.click()
    download = dl_info.value
    saved = tmp_path / download.suggested_filename
    download.save_as(saved)
    assert saved.stat().st_size > 0

    page.goto(f"{BASE}/upload")
    page.locator("#file-upload").set_input_files(saved)
    page.locator("#file-submit").click()
    expect(page.locator("#uploaded-files")).to_have_text(saved.name)

@pytest.mark.parametrize("clicks, reverse", [(1, False), (2, True)], ids=["asc", "desc"])
def test_table_sorts_by_last_name(page: Page, clicks, reverse):
    page.goto(f"{BASE}/tables")
    header = page.locator("#table1 thead").get_by_text("Last Name")
    for _ in range(clicks):
        header.click()
    last_names = page.locator("#table1 tbody tr td:nth-child(1)").all_inner_texts()
    assert is_sorted(last_names, reverse=reverse), last_names

@pytest.fixture
def auth_page(browser: Browser):
    context = browser.new_context(http_credentials={"username": "admin", "password":
"admin"})
    yield context.new_page()
    context.close()

def test_basic_auth(auth_page: Page):
    auth_page.goto(f"{BASE}/basic_auth")
    expect(auth_page.get_by_text("Congratulations!")).to_be_visible()
```

WHY IT'S HERO-LEVEL

Each pattern asserts an outcome: the saved file's size, the sorted names, the success text after basic auth.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **How do you handle JavaScript alerts in Playwright?**

Register a dialog handler *before* triggering it (`page.once("dialog", lambda d: d.accept())` , or `dismiss()` , or `accept("text")` for prompts), then click. Without a handler, Playwright auto-dismisses dialogs. The handler can also assert `dialog.message` and `dialog.type` .

Z **How do you interact with an element inside an iframe?**

Use `page.frame_locator("iframe selector")` to scope into the frame, then locate inside it. Chain `frame_locator` calls for nested frames. There's no "switch to frame" state like in Selenium, so each locator carries its frame path.

H **How do you handle a link that opens a new tab?**

Wrap the click in `with page.context.expect_page() as info:` (or `page.expect_popup()`), then `new_page = info.value` , and interact with it like any page. Both tabs stay usable, with no window switching.

H **How do you verify a file download?**

Use `with page.expect_download() as info: trigger()` , then `download = info.value` . Check `suggested_filename` , `save_as(tmp_path / ...)` , and validate the content (size, parse CSV/PDF text, checksum). If the download URL is known, fetching it via the API client and validating the bytes is often faster and more robust.

Z **Why avoid `sleep` / `wait_for_timeout` in UI tests?**

Fixed waits are both slow (they always wait the full time) and flaky (the app may need longer on a slow CI run). Wait for conditions instead: auto-waiting actions, `expect` assertions with explicit timeouts, URL/response waits. The only acceptable sleep is in retry backoff infrastructure, never in test logic.

H **Drag and drop doesn't work in your test. What do you try?**

First `source.drag_to(target)`. If the app uses custom mouse-event logic, use manual steps: `hover` source, `mouse.down()`, move to the target in steps (`mouse.move(x, y, steps=10)`), `mouse.up()`. Check whether it's HTML5 DnD or a JS library, verify with a trace, and assert the resulting state, not the motion.

🤖 AI ASSIST

When a test fails, paste the **error plus the relevant trace step description** into AI and ask for hypotheses ranked by likelihood. Then verify in the trace.

P05-06 · 🔵 HERO · 5H

Toolshop: auth state, hybrid API+UI, network mocking

App — Toolshop

Deliverable — `framework/pages/toolshop/`, `tests/ui/toolshop/`

📋 The task

1. **Login once, reuse everywhere:** a session fixture logs in via the UI, saves `storage_state` to a file, and a `customer_page` fixture opens pre-authenticated contexts. Measure the time saved.
2. **Hybrid test:** register a brand-new user **via the API** (P04-07 factory), log in via the **UI**, and verify the nav menu shows the name that `GET /users/me` returns.
3. **Wait for the network, not time:** search for "hammer" in the UI, capture the product-search API call with `page.expect_response`, and assert that the UI shows exactly the products the API returned.
4. **Mock the network:** use `page.route` to (a) return an empty product list → verify the empty-state UI, (b) return a 500 → verify the app doesn't break silently, and (c) block images to speed up tests.
5. Page objects: `ToolshopLoginPage`, `ToolshopHomePage` (search, product cards), `NavMenu` component.

`done when` Tick these off

- [ ] UI tests don't log in through the form except in the one session fixture
- [ ] The mocked tests are deterministic (they pass even if Toolshop's catalogue changes)

📘 User guide

`concepts` What you need to know

- `storage_state` : a JSON snapshot of cookies and localStorage. Load it into new contexts to start authenticated.
- **Hybrid testing:** use the fast API for *setup* and *verification of data*, and the UI only for the *behaviour under test*.
- `page.expect_response(predicate)` / `expect_request` : wait for a specific network call and inspect it.
- `page.route(pattern, handler)` : intercept requests: `route.fulfill(...)` (fake a response), `route.abort()` (block), `route.continue_(...)` (modify and pass), `route.fetch()` (call the real server then modify).

`deep dive` Going deeper

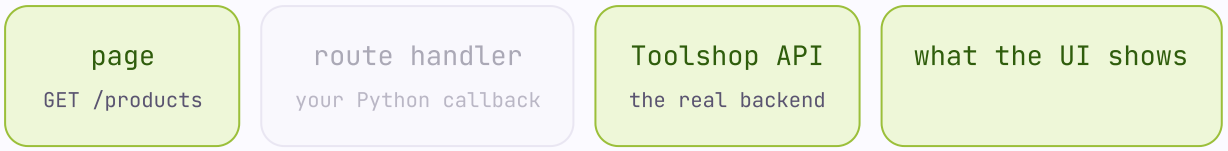
Why auth reuse matters: logging in through the UI can take 2–4 s per test. With 300 tests that's 10–20 minutes of pure overhead, and every login is a flake opportunity. Log in **once per worker**, save the state, and give each test a fresh context *with* that state: isolation *and* speed. Keep one dedicated test that exercises the login form itself.

Hybrid tests are the senior default. "Create a user via UI, then log in, then check the profile" tests three UI flows to verify one. "Create via API → act in UI → verify via API/DB" is faster, more stable and pinpoints failures. It's the "test at the lowest layer that can catch the bug" principle.

Mocking: power and danger. Mocks make edge cases deterministic (empty lists, 500s, slow responses, huge payloads) that are hard to create for real. But a mocked test can't catch backend integration bugs, and mocks drift from reality. Rules: mock to test **UI behaviour** for rare states, keep a real (unmocked) happy path, and build mock payloads from real responses (`route.fetch()` + modify) to stay realistic.

Wait for what matters. After typing a search, the UI updates when the API responds. `with page.expect_response(lambda r: "/products/search" in r.url) as resp:` waits exactly for that. It's deterministic and it gives you the API data to compare with the UI.

INTERACTIVE — WHAT PAGE.ROUTE DOES TO ONE REQUEST



No route: the request reaches the real backend, and the UI shows whatever the catalogue holds today. Keep one test like this.

Worked examples

LEVEL 1 — CRACK IT

🟢 Low: page objects and a UI login.

framework/pages/toolshop/login_page.py

```python
# framework/pages/toolshop/login_page.py
from playwright.sync_api import Page, expect

class ToolshopLoginPage:
    def __init__(self, page: Page) -> None:
        self.page = page

    def open(self) -> "ToolshopLoginPage":
        self.page.goto("/auth/login")
        return self

    def login(self, email: str, password: str) -> None:
        self.page.locator('[data-test="email"]').fill(email)
        self.page.locator('[data-test="password"]').fill(password)
        self.page.locator('[data-test="login-submit"]').click()
        expect(self.page.locator('[data-test="nav-menu"]')).to_be_visible()
```

SAY THIS

One page object for login; the form itself is exercised through the UI only in this one place.

LEVEL 2 — IMPRESS THEM

🔵 Medium: session storage state plus a hybrid test (API setup → UI → API verification).

tests/ui/toolshop/conftest.py

```python
# tests/ui/toolshop/conftest.py
import pytest
from framework.pages.toolshop.login_page import ToolshopLoginPage

@pytest.fixture(scope="session")
def customer_state(browser, settings, tmp_path_factory):
    path = tmp_path_factory.mktemp("auth") / "customer.json"
    context = browser.new_context(base_url=settings.toolshop_url)
    page = context.new_page()
    ToolshopLoginPage(page).open().login("customer@practicesoftwaretesting.com",
"welcome01")
    context.storage_state(path=str(path))
    context.close()
    return path

@pytest.fixture
def customer_page(browser, customer_state, settings):
    context = browser.new_context(storage_state=str(customer_state),
base_url=settings.toolshop_url)
    page = context.new_page()
    yield page
    context.close()
```

tests/ui/toolshop/test_hybrid_registration.py

```python
# tests/ui/toolshop/test_hybrid_registration.py
from playwright.sync_api import expect
from framework.data.factories import user_payload
from framework.pages.toolshop.login_page import ToolshopLoginPage

def test_api_registered_user_sees_own_name_in_ui(browser, http, settings):
    user = user_payload()
    r = http.post(f"{settings.toolshop_api_url}/users/register", json=user, timeout=10)
# API setup
    assert r.status_code in (200, 201), r.text

    context = browser.new_context(base_url=settings.toolshop_url)
    page = context.new_page()
    ToolshopLoginPage(page).open().login(user["email"], user["password"])
# UI behaviour
    expect(page.locator('[data-test="nav-menu"]')).to_contain_text(user["first_name"])
    context.close()
```

HIRE SIGNAL

Log in once per session, save `storage_state`, and give every test a fresh pre-authenticated context; set up data through the API.

LEVEL 3 — HERO MODE

🟣 High/Complex: network assertions and three kinds of mocks.

tests/ui/toolshop/test_network.py

```python
# tests/ui/toolshop/test_network.py
import json, re
from playwright.sync_api import Page, Route, expect

PRODUCTS_API = re.compile(r".*/products(\?.*)?$")
EMPTY_PAGE = {"current_page": 1, "data": [], "from": None, "last_page": 1, "per_page": 9,
"to": None, "total": 0}

def test_search_results_match_api(customer_page: Page):
    customer_page.goto("/")
    customer_page.locator('[data-test="search-query"]').fill("hammer")
    with customer_page.expect_response(lambda r: "/products/search" in r.url and r.ok) as
resp_info:
        customer_page.locator('[data-test="search-submit"]').click()
    api_names = sorted(p["name"] for p in resp_info.value.json()["data"])
    ui_names = customer_page.locator('[data-test="product-name"]')
    expect(ui_names).to_have_count(len(api_names))
    assert sorted(n.strip() for n in ui_names.all_inner_texts()) == api_names

def test_empty_catalogue_shows_no_cards(page: Page):
    page.route(PRODUCTS_API, lambda route: route.fulfill(status=200,
content_type="application/json",
                                                         body=json.dumps(EMPTY_PAGE)))
    page.goto("/")
    expect(page.locator('[data-test="product-name"]')).to_have_count(0)
    # Observe the real empty-state message and assert it here: that's part of the task

def test_server_error_is_not_silent(page: Page):
    page.route(PRODUCTS_API, lambda route: route.fulfill(status=500, body="boom"))
    page.goto("/")
    expect(page.locator('[data-test="product-name"]')).to_have_count(0)
    # What SHOULD the user see? If nothing tells them, that's a UX bug worth reporting.

def test_price_patched_from_real_response(page: Page):
    def patch_prices(route: Route) -> None:
        response = route.fetch()                          # real backend call
        body = response.json()
        for p in body["data"]:
            p["price"] = 0.01
        route.fulfill(response=response, json=body)
    page.route(PRODUCTS_API, patch_prices)
    page.route(re.compile(r".*\.(png|jpg|jpeg|webp|svg)$"), lambda r: r.abort())   #
faster: no images
    page.goto("/")
    expect(page.get_by_text("$0.01").first).to_be_visible()
```

NOTE

Confirm the endpoint paths and `data-test` names with codegen. Toolshop versions differ.

WHY IT'S HERO-LEVEL

`expect_response` waits for the exact API call and compares its data with the UI; `route()` makes empty lists, 500s and patched prices deterministic.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **How do you avoid logging in through the UI in every test?**

Authenticate once (per session or per worker) via UI or API, save the context's `storage_state` (cookies and localStorage), and create each test's context from it, so every test is fresh *and* pre-authenticated. Keep dedicated tests for the login flow itself. With multiple roles, keep one state file per role.

H **What is a hybrid API+UI test and why prefer it?**

A test that uses the API for setup (and often verification) and the UI only for the behaviour under test. It's faster, less flaky and has clearer failure localisation, because UI steps unrelated to the goal are removed. Example: create an order via the API, then verify the order-history page renders it correctly.

H **When do you mock network responses in UI tests? What are the risks?**

Use: rare or hard-to-produce states (empty lists, errors, timeouts, huge data, feature flags), third-party dependencies, and deterministic component-level UI tests. Risks: missing real integration bugs, and mocks drifting from the real API. Mitigate by keeping real end-to-end happy paths, deriving mocks from real responses (`route.fetch()` + patch), and contract-testing the API (P04-08).

H **How do you wait for a specific API call triggered by a UI action?**

Wrap the action in `with page.expect_response(predicate) as info:`, where the predicate matches the URL, method or status. Then `info.value.json()` gives the data, which you can compare with what the UI renders. This replaces arbitrary waits and doubles as an integration check.

SH **How do you test UI error handling for backend failures?**

Use route interception to inject 500s, timeouts (`route.abort("timedout")`), slow responses and malformed JSON per endpoint. Assert user-facing behaviour: a clear message, a retry option, no infinite spinner, no stale data shown as fresh, form input preserved, and errors logged or reported. Combine with chaos-style tests in lower environments.

SH **With 8 parallel workers and 3 roles, how do you manage authentication state?**

Create state per worker and role in a session fixture (so tokens aren't shared across processes, which avoids session-invalidation collisions), or generate once with a file lock if the backend allows concurrent sessions for one user. Better still, give each worker its own users (e.g. `customer_w3@…`) so server-side session limits or cart state can't collide.

🤖 AI ASSIST

Ask AI to list 10 backend failure modes worth simulating for a product-listing page. Implement 3 with `page.route` .

P05-07 · SUPER HERO · 5H

Debugging, cross-browser, mobile, visual checks and bug hunt

App — SauceDemo, Toolshop with-bugs

Deliverable — `tests/ui/test_visual.py`, `tests/ui/test_bug_hunt.py`, `docs/bugs/`, `docs/ui-debugging.md`

📋 The task

1. **Evidence on failure:** configure `--tracing retain-on-failure --screenshot only-on-failure --video retain-on-failure --output test-results`. Break a test deliberately and diagnose it **only** from the trace. Write the steps in `docs/ui-debugging.md`.
2. **Interactive debugging:** use `PWDEBUG=1` and `page.pause()` once each.
3. **DevTools and trace side by side:** run one Toolshop test headed with the browser's DevTools open (`--headed`, then F12 → **Network**, filter by *Fetch/XHR*). Find the API call behind a UI action (e.g. adding to the cart), and match it to the same request in the trace's **Network** tab: method, URL, status, timing and response body. Note in `docs/ui-debugging.md` which questions each tool answers better.
4. **Cross-browser:** run the suite on all three engines, and use `@pytest.mark.skip_browser` / `only_browser` where justified (with a reason).
5. **Mobile:** run login and checkout with `--device "iPhone 13"` and fix anything that breaks (menus, layout).
6. **Visual check:** compare the inventory page for `standard_user` vs `visual_user` with a Pillow pixel diff, masking dynamic regions. Keep a baseline under `tests/ui/__snapshots__/`.
7. **Bug hunt:** write tests that expose ≥3 bugs with `problem_user` / `error_user` / Toolshop **with-bugs** (strict xfail + a bug ID), and a bug report for each in `docs/bugs/` (the template is in P10-01).

`done when` Tick these off

- [ ] A trace from a failing run is attached to a PR comment, with your diagnosis
- [ ] `docs/ui-debugging.md` shows the same API call found in DevTools and in the trace, and when you'd use each
- [ ] The visual test fails for `visual_user` and passes for `standard_user`

- [ ] 3 bug reports exist, each reproducible from the test

📘 User guide

concepts What you need to know

- **Trace viewer:** a timeline of actions with DOM snapshots before and after, console, network, source and screenshots. Open with `playwright show-trace trace.zip`.
- **Debug modes:** `PWDEBUG=1` (Inspector, step through), `page.pause()`, `--headed --slowmo`.
- **Engines:** Chromium (Chrome/Edge), Firefox (Gecko), WebKit (Safari's engine).
- **Device emulation:** viewport, user agent, touch, device scale factor (`playwright.devices[...]`, `--device`).
- **Visual testing:** compare screenshots to a baseline, with thresholds and masking.
- **Accessibility (a11y):** automated rule checks (e.g. axe-core) catch about 30–40% of WCAG issues.

deep dive Going deeper

Debugging a failed CI test, fastest path: (1) read the error and failing step, (2) open the **trace**: look at the DOM snapshot *at* the failing action (was the element there? covered? different text?), (3) check the **network** tab (did an API call fail or come back slow?), (4) check the **console** (JS errors), (5) reproduce locally with the same browser and `--headed`, (6) then fix the root cause: locator, wait condition, data, or a real bug.

DevTools vs the trace: DevTools is *live*: you can poke the page, edit the DOM, replay a request, throttle the network and read the console while the app runs. The trace is a *recording*: it shows exactly what the test saw at each step, including the DOM snapshot at the failing action, which DevTools can't give you after the fact. Use DevTools while writing and exploring a test, and the trace when a test has already failed (especially in CI). Both show the network, and a UI bug is often an API bug in disguise: a 500 or a slow response behind a spinner that never ends.

Common flaky UI causes: racing a re-render (fix with `expect`), animations (auto-wait usually handles them), shared test data, hard-coded waits, reliance on order, third-party scripts, viewport-dependent layouts, timezone/locale differences, and unhandled dialogs.

Cross-browser strategy: run the full suite on Chromium (most users), and a smoke or critical subset on Firefox and WebKit per PR, with everything nightly. WebKit on Linux

approximates Safari but isn't identical, so real-Safari checks may need a device cloud.

Emulation vs real devices: emulation covers viewport, touch and user agent, which is good for responsive layout and flows. It doesn't reproduce real mobile performance, OS keyboards or real Safari quirks. Use a device cloud for a small real-device subset.

Visual testing gotchas: fonts, anti-aliasing and GPU differ across OS and machines, so **generate baselines and compare in the same Docker image** (P08). Mask dynamic areas (dates, ads), set thresholds, and review baseline updates like code.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: evidence-on-failure defaults.

pyproject.toml

```
# pyproject.toml → [tool.pytest.ini_options]
addopts = """-ra --strict-markers --base-url https://www.saucedemo.com
  --tracing retain-on-failure --screenshot only-on-failure --video retain-on-failure
  --output test-results"""
```

```
uv run pytest tests/ui -k checkout --browser chromium                # break a locator first
uv run playwright show-trace test-results/*/trace.zip
PWDEBUG=1 uv run pytest tests/ui/saucedemo/test_login.py::test_standard_user_can_login
```

SAY THIS

Traces, screenshots and video are kept only for failures, so evidence costs nothing on green runs.

LEVEL 2 — IMPRESS THEM

🔵 Medium: cross-browser and mobile.

```
uv run pytest tests/ui -m smoke --browser chromium --browser firefox --browser webkit
uv run pytest tests/ui/saucedemo --device "iPhone 13" --browser webkit   # is_mobile isn't supported on Firefox
```

```python
import pytest

@pytest.mark.skip_browser("firefox")        # reason: native <select> styling differs;
covered in chromium/webkit
def test_sort_dropdown_visual_state(logged_in_page): ...

@pytest.mark.only_browser("chromium")       # reason: uses CDP-only performance metrics
def test_page_weight_budget(logged_in_page): ...
```

HIRE SIGNAL

The critical path runs on all three engines; a browser is skipped only with a written reason.

LEVEL 3 — HERO MODE

🟣 High/Complex: a visual diff with masking and baselines, plus a bug-exposing test.

tests/ui/test_visual.py

```python
# tests/ui/test_visual.py      (uv add --dev pillow)
import os
from pathlib import Path
import pytest
from PIL import Image, ImageChops
from framework.pages.saucedemo.login_page import LoginPage

SNAP = Path(__file__).parent / "__snapshots__"
UPDATE = os.getenv("UPDATE_SNAPSHOTS") == "1"

def diff_ratio(a: Path, b: Path) -> float:
    img_a, img_b = Image.open(a).convert("RGB"), Image.open(b).convert("RGB")
    if img_a.size != img_b.size:
        return 1.0
    diff = ImageChops.difference(img_a, img_b).convert("L").point(lambda px: 255 if px > 16
else 0)
    return sum(1 for px in diff.getdata() if px) / (img_a.width * img_a.height)

@pytest.mark.only_browser("chromium")
@pytest.mark.parametrize("user, should_match", [("standard_user", True), ("visual_user",
False)])
def test_inventory_visual(page, tmp_path, browser_name, user, should_match):
    page.set_viewport_size({"width": 1280, "height": 900})
    inventory = LoginPage(page).open().login(user)
    actual = tmp_path / f"{user}.png"
    page.screenshot(path=str(actual), full_page=True, mask=[inventory.header.badge],
animations="disabled")
    baseline = SNAP / f"inventory-{browser_name}.png"
    if UPDATE and user == "standard_user":
        SNAP.mkdir(exist_ok=True); baseline.write_bytes(actual.read_bytes());
pytest.skip("baseline updated")
    ratio = diff_ratio(baseline, actual)
    assert (ratio < 0.001) == should_match, f"{user}: {ratio:.4%} pixels differ"
```

tests/ui/test_bug_hunt.py

```
# tests/ui/test_bug_hunt.py
import pytest
from framework.pages.saucedemo.login_page import LoginPage

@pytest.mark.xfail(strict=True, reason="BUG-001: problem_user sees the same image for every
product")
def test_product_images_are_distinct_for_problem_user(page):
    LoginPage(page).open().login("problem_user")
    srcs = page.locator(".inventory_item img").evaluate_all("imgs => imgs.map(i => i.src)")
    assert len(set(srcs)) == len(srcs), f"duplicate images: {srcs}"
```

NOTE

Baselines depend on OS and fonts: generate them **inside the Docker image** you use in CI (P08-01) with `UPDATE_SNAPSHOTS=1`. Optional stretch: an accessibility scan with the `axe-playwright-python` package (check its docs) on the login page.

WHY IT'S HERO-LEVEL

The visual check masks dynamic regions and uses a threshold; the bug-hunt test is a strict xfail with a bug ID.

try it diff_ratio() on generated images + ADDED

The comparison function is plain Pillow, so it runs here: a 4-pixel shift fails the 0.1% threshold, a barely visible colour change passes, and any size change counts as 100% different.

```python
# + added: the diff_ratio() from tests/ui/test_visual.py, on three generated images
from pathlib import Path

from PIL import Image, ImageChops, ImageDraw


def diff_ratio(a: Path, b: Path) -> float:                # exactly as in test_visual.py
    img_a, img_b = Image.open(a).convert("RGB"), Image.open(b).convert("RGB")
    if img_a.size != img_b.size:
        return 1.0
    diff = ImageChops.difference(img_a, img_b).convert("L").point(lambda px: 255 if px > 16
else 0)
    return sum(1 for px in diff.getdata() if px) / (img_a.width * img_a.height)


def card(x: int, colour) -> Image.Image:
    img = Image.new("RGB", (200, 100), "white")
    ImageDraw.Draw(img).rectangle([x, 10, x + 50, 40], fill=colour)
    return img


card(10, (0, 0, 128)).save("baseline.png")
card(14, (0, 0, 128)).save("moved.png")                   # the same box, 4 px to the right
card(10, (0, 0, 140)).save("tinted.png")                  # a colour change too small to
see
card(10, (0, 0, 128)).resize((201, 100)).save("resized.png")

for name in ("baseline", "moved", "tinted", "resized"):
    ratio = diff_ratio(Path("baseline.png"), Path(f"{name}.png"))
    print(f"{name:9} {ratio:8.4%}  {'matches' if ratio < 0.001 else 'DIFFERS'} (threshold
0.1%)")
```

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **A UI test fails only in CI. How do you debug it?**

Pull the artifacts: trace, screenshot, video, logs. In the trace, check the DOM snapshot at the failing step, network calls (slow or failed?) and console errors. Compare the environment: headless vs headed, viewport, browser version, timezone/locale, data, CPU speed. Reproduce with the same Docker image and flags. Fix the root cause (wait condition, locator, data isolation, or a real bug), never add sleeps or blanket retries.

H **What does the Playwright trace contain?**

An action timeline with before/after DOM snapshots (inspectable like DevTools), screenshots, network requests and responses, console logs, errors, the source line of each step, and timing. It's usually enough to diagnose a failure without re-running.

H **What's your cross-browser strategy?**

Risk- and usage-based: the full suite on the main engine (usually Chromium), smoke or critical paths on Firefox and WebKit per PR, everything on all engines nightly, and a small real-device/Safari subset via a device cloud before releases. Use analytics to decide the matrix, and only skip browsers with explicit reasons.

SH **Emulated mobile vs real devices?**

Emulation (viewport, touch, UA, DPR) is fast, free and good for responsive layout and functional flows in CI. It doesn't capture real device performance, OS keyboards and permissions, real mobile Safari quirks, or network conditions. Use emulation broadly and real devices selectively for high-risk flows.

SH **What makes visual regression testing hard, and how do you make it reliable?**

Rendering differs by OS, fonts, GPU and browser version, dynamic content changes, and small diffs create noise. Make it reliable by comparing in a pinned container, masking dynamic regions, disabling animations, using sensible thresholds, scoping to key components rather than whole pages, and making baseline updates reviewable (diff images in PRs). Consider a hosted visual tool for team review workflows.

SH **How do you automate accessibility testing, and what are its limits?**

Run an axe-core scan on key pages and states in CI (gate on serious/critical violations), and use role-based locators, which fail when accessible names are missing. Limits: automated rules catch a minority of WCAG issues. Keyboard navigation, focus order, screen-reader announcements and content clarity need manual or assisted testing.

🤖 AI ASSIST

- Give AI a trace's **action list and error** (no secrets) and ask for a ranked diagnosis. Compare it with yours and note where AI was wrong.
- Use AI to polish bug reports' wording, but **never** to invent reproduction steps you didn't verify.

P05-08 · 🟣 SUPER HERO · 3H

Selenium to Playwright migration

App — SauceDemo (`legacy-suite/selenium/`)

Deliverable — `docs/selenium-migration.md` , `tests/ui/migrated/` , `scripts/parity.py`

📋 The task

1. Run the Selenium suite and record its runtime: `uv run --with selenium python -m pytest legacy-suite/selenium -v --junitxml=test-results/selenium.xml` (it needs Chrome; Selenium Manager downloads the matching driver).
2. Write a **concept map** from Selenium to Playwright in `docs/selenium-migration.md` (🟢 example), including at least three things that *don't* translate one-to-one.
3. **Migrate** the 5 tests into `tests/ui/migrated/` , using your page objects where they fit. **Keep the test names**, so each old test maps to its replacement.
4. **Prove parity:** run both suites with `--junitxml` and compare them with `scripts/parity.py` (🟣 example). Then break one expectation in both suites and check that they fail the same way. Compare the runtimes.
5. Write a **migration plan for a real suite of 800 Selenium tests**: order of migration, parity metrics, running both suites in CI during the transition, decommission criteria, risks, and training.

done when Tick these off

- [ ] The 5 migrated tests pass, with no sleeps and no XPath

- [ ] `scripts/parity.py` reports all 5 at parity, and your doc shows the runtime of both suites
- [ ] The 800-test plan has milestones, exit criteria, and at least one reason a test should be *pushed down* to the API layer instead of migrated

📘 User guide

`concepts` What you need to know

- **WebDriver:** the W3C protocol Selenium uses: a driver process (chromedriver, geckodriver) translates commands for the browser. **WebDriver BiDi** is its newer, two-way successor.
- **Playwright's model:** one connection per browser, with isolated **contexts** (cheap, incognito-like profiles) and **pages** inside them. Actions auto-wait; assertions retry.
- **Implicit vs explicit waits:** Selenium's implicit wait applies to every `find_element`; explicit waits (`WebDriverWait` + `expected_conditions`) wait for a specific condition. Mixing them causes unpredictable timeouts. Playwright replaces both with auto-waiting and `expect`.
- **Parity testing:** running old and new suites side by side and comparing results test by test until you trust the new one.

`deep dive` Going deeper

Re-express intent; don't translate lines. A line-by-line port carries the old habits across: explicit waits become pointless `wait_for_selector` calls, XPaths survive, sleeps survive. Read each Selenium test for *what it protects*, then write the Playwright test the way you would from scratch: user-facing locators, `expect`, fixtures and page objects.

What doesn't translate one-to-one:

- **Waiting:** `WebDriverWait(...).until(EC.visibility_of_element_located(...))` mostly *disappears*, because Playwright waits before every action and `expect` retries.
- **Frames, windows and alerts:** `switch_to.frame` becomes `frame_locator` (no global "current frame"); window handles become `context.expect_page()`; `switch_to.alert` becomes a `page.on("dialog")` handler registered *before* the action.
- **Sessions:** a new WebDriver per test is slow, so Selenium suites often share one; in Playwright a new context per test is cheap, so isolation comes for free.
- **Grids:** Selenium Grid and cloud grids work with WebDriver; Playwright runs its own browsers (or connects to a remote Playwright server), so infrastructure plans change.

Why teams migrate: faster, more stable tests (auto-waiting), better debugging (traces, codegen, the inspector), built-in network interception, and one API for three engines. **Why some shouldn't:** a large, *stable* Selenium suite with a working grid has little to gain; mobile-native testing needs Appium anyway; and a Java-heavy team may prefer to stay with its ecosystem. Selenium 4 closed part of the gap (relative locators, BiDi, Selenium Manager), so be fair in the comparison.

Migration economics: every migrated test costs time, so migrate in order of *value × pain*: flaky, high-value tests first, and stable, low-value ones last (or never). Many UI tests turn out to be checking business rules, and those should be **pushed down** to API tests rather than migrated. A migration is the cheapest moment to fix the pyramid.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: the concept map.

SELENIUM (PYTHON)	PLAYWRIGHT (PYTHON)
`webdriver.Chrome()` per test	`browser` per session,
`driver.get(url)`	`page.goto(url)`
`find_element(By.ID, "user-name")`	`page.get_by_placeho` `page.locator('[data-`
`find_element(By.XPATH, "//div[text()='X']/ancestor::…//button")`	`page.locator(".inver`
`implicitly_wait(10)` + `WebDriverWait` + `EC`	Auto-waiting actions +
`element.text == "Products"`	`expect(locator).to_`
`Select(el).select_by_value("lohi")`	`locator.select_optio`
`driver.switch_to.frame(...)`	`page.frame_locator(`
`driver.switch_to.alert.accept()`	`page.on("dialog", la`
`driver.save_screenshot(...)` on failure	`--screenshot only-o`

SAY THIS

Map concepts, not lines: explicit waits disappear, frames become `frame_locator`, alerts become handlers registered first.

LEVEL 2 — IMPRESS THEM

🔵 Medium: one test, before and after.

```python
# Before: legacy-suite/selenium/test_saucedemo_selenium.py
def test_login_locked_out(driver):
    login(driver, "locked_out_user")
    error = WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located((By.CSS_SELECTOR, "[data-test='error']"))
    )
    assert "locked out" in error.text
```

```python
# After: tests/ui/migrated/test_saucedemo_migrated.py
from playwright.sync_api import Page, expect

from framework.pages.saucedemo.login_page import LoginPage


def test_login_locked_out(page: Page) -> None:
    login = LoginPage(page).open()
    login.username.fill("locked_out_user")
    login.password.fill("secret_sauce")
    login.submit.click()
    expect(login.error).to_contain_text("locked out")
```

The explicit wait, the `By` import and the manual `.text` read all disappear; `expect` waits *and* checks. (It doesn't call `login.login()`, because that method waits for the products page, which a locked-out user never reaches.)

HIRE SIGNAL

The migrated test re-expresses the intent: no explicit wait, no `By` import — `expect` both waits and checks.

LEVEL 3 — HERO MODE

🟣 High/Complex: prove parity test by test.

scripts/parity.py

```python
# scripts/parity.py: compare two JUnit XML reports test by test
"""Usage: python scripts/parity.py old.xml new.xml"""

import sys
import xml.etree.ElementTree as ET


def results(path: str) -> dict[str, str]:
    outcome = {}
    for case in ET.parse(path).getroot().iter("testcase"):
        name = case.get("name", "").split("[")[0]  # drop pytest parameters such as
[chromium]
        if case.find("failure") is not None or case.find("error") is not None:
            outcome[name] = "failed"
        elif case.find("skipped") is not None:
            outcome[name] = "skipped"
        else:
            outcome[name] = "passed"
    return outcome


def main(old_path: str, new_path: str) -> int:
    old, new = results(old_path), results(new_path)
    problems = 0
    for name in sorted(old.keys() | new.keys()):
        before, after = old.get(name, "missing"), new.get(name, "missing")
        mark = "✅" if before == after == "passed" else "❌"
        problems += mark == "❌"
        print(f"{mark} {name:40} {before:>8} → {after}")
    print(f"\n{len(old)} old, {len(new)} new, {problems} not at parity")
    return 1 if problems else 0


if __name__ == "__main__":
    sys.exit(main(sys.argv[1], sys.argv[2]))
```

```
uv run --with selenium python -m pytest legacy-suite/selenium --junitxml=test-
results/selenium.xml
uv run pytest tests/ui/migrated --junitxml=test-results/playwright.xml
uv run python scripts/parity.py test-results/selenium.xml test-results/playwright.xml
```

When this guide was written, the Selenium suite took about 8 s and a straightforward Playwright port about 4 s for the same five tests. At 800 tests, run in CI on every PR, that difference is the business case.

Plan skeleton for 800 tests (put the details in your doc):

STAGE	WHAT HAPPENS	EXIT CRITERION
0. Inventory	Tag every Selenium test with value, flake rate and runtime; mark rule checks to push down to the API layer	Every test tagged

STAGE	WHAT HAPPENS	EXIT CRITERION
1. Foundations	Playwright fixtures, page objects, CI job, reporting and traces	A pilot of 20 tests at parity for 2 weeks
2. Strangle	New tests only in Playwright; migrate the flaky, high-value tests first; run both suites in CI	Parity report green; flake rate lower than the old suite
3. Decommission	Remove migrated Selenium tests weekly; switch the required CI check	The Selenium job is removed and the grid cost saved

WHY IT'S HERO-LEVEL

`parity.py` compares the two JUnit reports test by test, so switching suites is a decision based on evidence.

try it parity.py on two small reports + ADDED

```
# + added: parity.py on two small JUnit reports
from scripts.parity import main

SELENIUM = """<testsuite name="selenium">
  <testcase name="test_login_standard_user"/>
  <testcase name="test_login_locked_out"/>
  <testcase name="test_add_to_cart"/>
  <testcase name="test_sort_low_to_high"><skipped message="flaky"/></testcase>
</testsuite>"""
PLAYWRIGHT = """<testsuite name="playwright">
  <testcase name="test_login_standard_user[chromium]"/>
  <testcase name="test_login_locked_out[chromium]"/>
  <testcase name="test_add_to_cart[chromium]"><failure message="badge never showed 1"/>
</testcase>
  <testcase name="test_sort_low_to_high[chromium]"/>
</testsuite>"""

for name, xml in (("selenium.xml", SELENIUM), ("playwright.xml", PLAYWRIGHT)):
    with open(name, "w") as f:
        f.write(xml)
print("exit code:", main("selenium.xml", "playwright.xml"))
```

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **How do Selenium and Playwright differ in the way they wait for elements?**

In Selenium you choose the waiting: implicit waits on every `find_element`, or explicit `WebDriverWait` with expected conditions, and mixing them causes surprises. Playwright auto-waits before every action (attached, visible, stable, enabled) and its `expect` assertions retry until they pass or time out, so most explicit waiting code disappears.

H **How would you migrate a large Selenium suite to Playwright?**

Inventory and tag the tests by value, flakiness and runtime; build the Playwright foundations and pilot a small set to parity; then strangle: new tests only in Playwright, migrate the flaky high-value tests first, push rule checks down to API tests, run both suites in CI with a parity report, and delete Selenium tests as their replacements prove themselves. Finish by switching the required check and decommissioning the grid. Measure runtime, flake rate and maintenance time throughout.

H **What doesn't translate one-to-one from Selenium to Playwright?**

Waits (mostly disappear), frames (`frame_locator` instead of switching), new windows (`context.expect_page()` instead of window handles), dialogs (a handler registered before the action), session management (a cheap new context per test instead of a shared driver) and grid infrastructure. Porting line by line keeps the old workarounds, so re-express each test's intent instead.

SH **When would you advise against migrating?**

When the existing suite is stable, fast enough and cheap to maintain; when the team's skills and ecosystem are strongly tied to Selenium (Java, an established grid, Appium for mobile); or when the effort would be better spent pushing UI tests down to the API layer. Recommend based on measured pain (flakiness, runtime, maintenance hours) versus migration cost, not on tool fashion.

🤖 AI ASSIST

AI is good at mechanical translation. Ask it to port one Selenium test, then check the output against the concept map: did it keep a `WebDriverWait`-style wait or an XPath "to be safe"? Those are the habits you're migrating away from.

🏁 PHASE CHECKPOINT — P05

Prove it without help

⏱️ Live-coding self-test (45 min, no AI)

from an empty repo, write a SauceDemo test that logs in, adds 2 items by name, checks out and verifies totals, using a small POM, `expect` assertions and no sleeps.

- [ ] Done from memory, without AI, inside 45 minutes

45-MINUTE TIMER

45:00

mock interview

AI mock interviewer prompt

```
You are interviewing me for a Playwright (Python) automation role. Ask 8 questions,
one at a time: locators, auto-waiting, flaky tests, POM design, iframes/tabs/dialogs,
auth state reuse, network mocking, CI debugging with traces. Then give me one
scenario: "this test passes locally but fails 1 in 10 runs in CI" and evaluate my
debugging approach. Score 1-5 each, with senior-level additions.
```

Record your results in `interview-notes/P05.md`.

BEYOND P05 — + ADDED

🧩 Four gaps worth closing

P05 covers the Playwright you need every day. These four features of the version this repo pins (1.63) come up quickly in real suites and in interviews. They need a browser, so copy them into your repo to run.

01 Playwright's own HTTP client

P05-06's hybrid test uses the `requests`-based `http` fixture. Playwright also ships an **APIRequestContext**: `playwright.request.new_context()` for standalone calls, and `page.request`, which shares the page's cookies — handy for calling the API as the user the browser just logged in as.

tests/ui/conftest.py

```python
# tests/ui/conftest.py: Playwright's own HTTP client (+ added)
import pytest
from playwright.sync_api import APIRequestContext, Playwright


@pytest.fixture(scope="session")
def toolshop_api(playwright: Playwright, settings) -> APIRequestContext:
    context = playwright.request.new_context(base_url=settings.toolshop_api_url)
    yield context
    context.dispose()


def test_products_endpoint_answers(toolshop_api: APIRequestContext):
    response = toolshop_api.get("/products")
    assert response.ok, response.text()
    assert response.json()["data"], "expected at least one product"
```

02 Every way to wait without sleeping

The P05-05 waiting toolkit, as code: a global `expect` timeout, one longer assertion, waiting for a state, a URL, or the API call itself.

```python
from playwright.sync_api import expect

expect.set_options(timeout=10_000)                              # raise the default for the whole run (e.g. in conftest.py)
expect(page.locator("#finish")).to_have_text("Hello World!", timeout=15_000)   # or for one assertion only
page.locator(".spinner").wait_for(state="hidden")           # wait for a state, with no assertion
page.wait_for_url("**/inventory.html")                          # wait for navigation
with page.expect_response("**/products/search*") as info:    # wait for the API call itself
    page.get_by_role("button", name="Search").click()
```

03 Control the clock

Time-dependent UI (session expiry, countdowns, "posted 5 minutes ago") is miserable to test with real time. `page.clock` lets the test set the date and fast-forward timers instantly.

```python
import datetime as dt

from playwright.sync_api import Page, expect


def test_session_timeout_warning(page: Page):
    """Illustrative: an app that warns 30 minutes into a session. SauceDemo has no such dialog."""
    page.clock.install(time=dt.datetime(2026, 1, 5, 9, 0))   # take control of Date, timers and intervals
    page.goto("/")
    page.clock.fast_forward("29:00")                            # 29 minutes pass instantly
    expect(page.get_by_role("dialog")).to_contain_text("about to expire")
```

04 Assert the accessibility tree

`to_match_aria_snapshot` compares a region's accessibility tree — roles and names, not pixels — with a template. It catches a lost label or a button that stopped being a button, which a visual diff can miss.

```python
from playwright.sync_api import Page, expect


def test_login_form_structure(page: Page):
    page.goto("/")
    # generate the real template with codegen's "Assert snapshot" tool, then trim it to
what matters
    expect(page.locator("form")).to_match_aria_snapshot("""
      - textbox "Username"
      - textbox "Password"
      - button "Login"
    """)
```

H **Aria snapshot vs visual diff?**

A visual diff checks how the page *looks* and is sensitive to fonts and rendering. An aria snapshot checks how it's *structured* for assistive tech and is stable across machines. They answer different questions; the aria snapshot is cheaper to keep green and doubles as an accessibility check.

FIELD GUIDE

🎓 Pattern recognition

The symptom → what to reach for.

`"it fails 1 run in 10"`

a snapshot assertion racing the UI

use expect; read the trace

`strict mode violation`

scope the locator

filter(has_text=…) inside a container

`a refactor broke 40 tests`

CSS/XPath locators

role, label or test id instead

`passes alone, fails in the suite`

shared page or data

a fresh context and own data per test

`the download or dialog is never seen`

expect, then trigger

register before the click

`every test spends 3 s logging in`

storage_state

log in once per worker

`can't reproduce "empty catalogue"`

page.route fulfill

keep a real happy path too

`a CI-only failure`

the trace first

DOM, network, console at the failing step

`a UI test checks a pricing rule`

push it down to the API

keep UI tests for UI behaviour

THE GOLDEN RULE

Wait for a condition, never for time.
Every sleep is a guess that's wrong in both directions.